There is a major disadvantage in linking library routines into the executable code. This is because every program using a routine has to have its own copy. This increases the storage space requirement for the executable file. It also increases memory usage when more than one process uses the routine.

The alternative is to use a routine from a dynamic linked library (DLL). When a DLL routine is available the executable code just requires a small piece of code to be included. This allows it to link to the routine, which is stored separately in memory, when execution of the program needs it. Many processes can be linked to the same routine. This has the advantage that the executable files for all programs need less storage space. Memory requirement is also minimised. Another advantage is that if a new version of the routine becomes available it can be loaded into memory so that any program using it is automatically upgraded.

The main disadvantage of using a DLL is that the program is relying on the routine being available and performing the expected function. If for some reason the DLL becomes corrupted or a new version has bugs not yet discovered the program will fail or produce an erroneous result. The user running the program will find it difficult to establish what needs to be done to get the program to run without error.

8.05 Language translators

The use of an assembler for translating a program written in assembly language has been discussed in Chapter 6 (Sections 6.02, 6.03 & 6.04). This chapter will introduce the translators that are used to translate a program written in a high-level procedural language.

Compilers and interpreters

The starting point for using either a compiler or an interpreter is a file containing source code, which is a program written in a high-level language.

For an interpreter the following steps apply.

1. The interpreter program, the source code file and the data to be used by the source code program are all made available.
2. The interpreter program begins execution.
3. The first line of the source code is read.
4. The line is analysed.
5. If an error is found, this is reported and the interpreter program halts execution.
6. If no error is found, the line of source code is converted to an intermediate code.
7. The interpreter program uses this intermediate code to execute the required action.
8. The next line of source code is read and Steps 4–8 are repeated.

For a compiler the following steps apply.

1. The compiler program and the source code file are made available but no data is needed.
2. The compiler program begins execution.
3. The first line of the source code is read.
4. The line is analysed.
5. If an error is found this is recorded.

6 If no error is found the line of source code is converted to an intermediate code.
7 The next line of source code is read and Steps 4–7 are repeated.
8 When the whole of the source code has been dealt with one of the following happens.
   - If no error is found in the whole source code the complete intermediate code is converted into object code.
   - If any errors are found a list of these is output and no object code is produced.

Execution of the program can only begin when the compilation has shown no errors. This can take place automatically under the control of the compiler program if data for the program is available. Alternatively, the object code is stored and the program is executed later with no involvement of the compiler.

Discussion Point:

What type of facility for language translation are you being provided with? Does your experience of using it match what has been described here?

The advantages and disadvantages to a programmer of creating interpreted or compiled programs.

- An interpreter has advantages when a program is being developed because errors can be identified as they occur and corrected immediately without having to wait for the whole of the source code to be read and analysed.
- An interpreter has a disadvantage in that during a particular execution of the program, parts of the code which contain syntax errors may not be accessed so if errors are still present, they are not discovered until later.
- An interpreter has a disadvantage when a program is error free and is distributed to users because the source code has to be sent to each user.
- A compiler has the advantage that an executable file can be distributed to users, so the users have no access to the source code.

The advantages and disadvantages to the user of interpreted or compiled programs.

- For an interpreted program, the interpreter and the source code have to be available each time that an error-free program is run.
- For a compiled program, only the object code has to be available each time that an error-free program is run.
- Compiled object code will provide faster execution than is possible for an interpreted program.
- Compiled object code is less secure because it could contain a virus.

Whether an interpreter or a compiler is used, a program can only be run on a particular computer with a particular processor if the interpreter or compiler program has been written for that processor.

If there is an option available the choice of an interpreter is justified when a program is being developed because:

- one error in a program can lead to several other errors occurring
- an interpreter can detect and correct an early error so limiting subsequent ones
- the debugging facilities provided in association with the interpreter speed this process.

The choice of a compiler is justified when the programmer is confident that the program is as near error-free as possible because:

- an executable file can be created
- this can be distributed for general use
- execution of the program will be faster than if an interpreter were used.

Java

When the programming language Java was created, a different philosophy was applied to how it should be used. Each different type of computer has to have a Java Virtual Machine created for it. Then when a programmer writes a Java program this is compiled first of all to create what is called Java Byte Code. When the program is run, this code is interpreted by the Java Virtual Machine. The Java Byte Code can be transferred to any computer that has a Java Virtual Machine installed.

8.06 Features found in a typical Integrated Development Environment (IDE)

Whatever language is used for writing source code and whatever compiler or interpreter is being used there will be one or more IDEs available to assist the programmer. This section discusses the types of feature that should be provided by an IDE.

Prettyprinting

Prettyprint refers to the presentation of the program code typed into an editor. For example, the Python IDLE (see Figure 8.02) automatically colour-codes keywords, built-in function calls, comments, strings and the identifier in a function header. In addition, indentation is automatic.

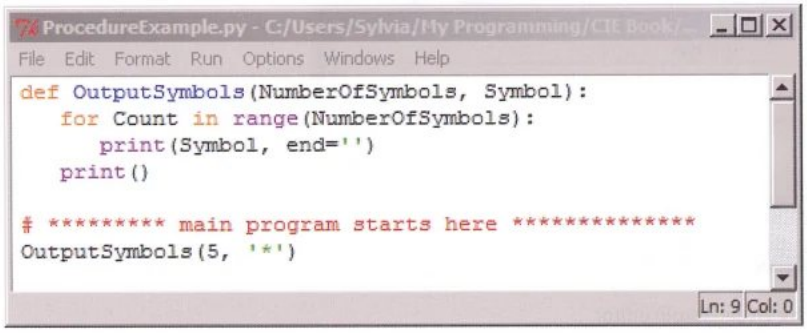


Figure 8.02 Prettyprint in the Python IDLE

Context-sensitive prompts

This feature displays hints (or a choice of keywords) and available identifiers that might be appropriate at the current insertion point of the program code. Figure 8.03 shows an example of the Visual Studio editor responding to text typed in by the programmer.